Pocket Cube Lab Overview.

This document contains excerpts from the pocket_cube lab. In this lab, I present the students with a simulation of a pocket cube and challenge them to solve it

The lab described an explanatory markdown "readme.md" file and comments in the pocket_cube.py file. The relevant comments are included here.

Readme.md

Pocket Cube Lab:

In this lab, we will explore Breadth First Search (BFS) and Dijkstra's algorithm using the Pocket Cube, a smaller version of a Rubik's cube. See comments in pocket_cube.py for a description.

In class, we covered BFS and Dijkstra's algorithm. However, the small examples in the notes seemed a little artificial becasue the graphs were held entirely in memory. The power of these algorithms is that they can be applied to large graphs.

The graph associated with the pocket cube has a vertex for every state of the cube and an edge when the states can be related by elementary moves, like twisting a face. The set of states forms a structure called a group. Generally speaking, a group is a set of transformations on an object. The elementary moves are called generators of the group whenever each configuration can be reached by applying the elementary moves. This occurs iff the graph is connected. Graphs whose vertices are a group and whose edges are generators are called Cayley graphs. They have nice symmetrical properties. Any configuration of the pocket cube could be considered as the solved state and the puzzle would still be the same.

These graphs are defined implicitly by the neighborhood function. This is a common situation. So the hope is the techniques needed to solve the pocket cube are generally applicable to a variety of problems.

The lab will consist of two parts:

1. Implement breadth-first search (BFS) and dijkstra's algorithm to solve small scrambles (length <5). You will notice that the implementations of these algorithms from the notes are inadequate. You should use a profiler to improve the performance of your implementations and use higher-order functions to make the algorithms flexible enough to allow a variety of approaches to a solution.
2. Solve the pocket cube. Ideally, with the shortest solution according to the costs of each move, which can be customized in the code. The ideas is that we can empircally time how long it takes someone to perform each

move, then use these empirical times to design an optimal solution for them. You will notice that the number of configurations is too large to solve directly using BFS or dijkstra's algorithm. So you should use BFS or dijkstra's algorithm as steps within a larger algorithm to solve the cube. This larger algorithm could involve many ideas, such as: -using the number of correctly placed/oriented cubies as an estimate of how solved the cube is. -taking advantage of invariants to expedite the search, like the amount of twist. -using commutators (sequences like ['F', 'D', 'Fp' 'Dp']) -use ideas from the standard solutions on Jaap's.

We may give significant hints on Nov. 11.

This repo contains the following files:

```
pocket_cube.py . . . . Contains the PocketCube class that simulates the pocket cube.
examples.py . . . . . .Contains some examples from which I wrote tests.
readme.md . . . . . .This file
utils . . . . . .Contains a decorator that stops your code if it is slow.
tests.py . . . . . Contains tests.
```

You should create a file called pocket_cube_solver.py and write your solving methods there. You may modify any of the prewritten code.

EXCERPT FROM pocket_cube.py

The pocket cube is a 2x2x2 variant of the Rubik's cube. It is equivalent to just the corners of the Rubik's cube. This file contains the code for our PocketCube class, which simulates the pocket cube.

The pocket cube consists of 8 "cubies." Each cubie can be oriented in any of 3 ways. If we specify the orientation of 7 cubies, then the last cubie's orientation is determined by the rest. By "state" we mean the position and orientation of each cubie. This means that there are 8!x3^7=88,179,840 states. You might see the claim that there are 3,674,160 states. This is because there are 24 orientations of the entire cube. But we are not accounting for these 24 orientations in this assignment.

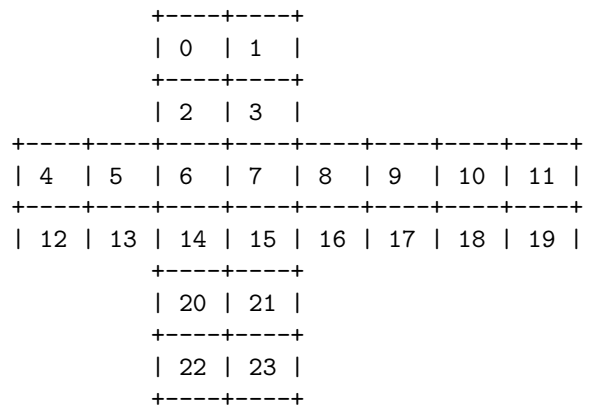
Further information can be found on Jaap's puzzle page: https://www.jaapsch.net/puzzles/cube2.htm

The pocketcube is a cube, and so has 6 facex. We denote the faces Front (F), Back(B), Right(R), Left(L), Up(U), Down(D). The notation is standard from David Singmaster's original notes on the Rubik's cube. The allowable moves consist of rotating a face of the cube clockwise 90, 180, or 270 degrees. To denote a move, we name the face to turn, followed by a suffix. If the face is alone, it is a 90 degree turn clockwise. If the face is followed by "p" (for prime), we turn the face 270 degrees (or 90 degrees counterclockwise). If the face is followed by "2", we turn the face 180 degrees. For example, "F" stands for rotating the front face clockwise. "F2" stands for rotating the front face counterclockwise.

There are multiple ways to quantify the smallest solution to the solved cube from a given state. Quarter Turn Metric (QTM): Each 90 degree or 270 degree turn counts as 1 move. A 180 degree turn counts as 2 moves. Half Turn Metric (HTM): Each allowable move counts as 1 move General metrics: (ALT): Assume we empirically determine how long it takes to perform each move. Assume for simplicity that each move and its inverse take the same amount of time. Assume the amount of time each move takes is independent of the previous and subsequent moves.) It is claimed that the pocket cube can be solved from any state in 11 moves using HTM and 14 moves using QTM. But we consider global orientation to be different, so for us it should take at most 15 moves using HTM and 22 using QTM.

We will use Dijkstra's algorithm and Breadth First Search (BFS) to solve the pocket cube. The number of states is too large to solve this directly, so we will need a greedy heuristic or some other trick.

To represent a state of the pocket cube, we will represent it by a list of length 24. The numbers in the list represent the faces of the cubies according to the following diagram, drawn by ChatGPT.



We also have a system for naming the cubies. For a given state, we name the 8 cubies of that state as cubie currently in the Back Top Left: (0,0,0) Back Top Right: (0,0,1) Back Down Left: (0,1,0) Back Down Right: (0,1,1) Front Top Left: (1,0,0) Front Top Right: (1,0,1) Front Down Left: (1,1,0) Front Down Right: (1,1,1)